

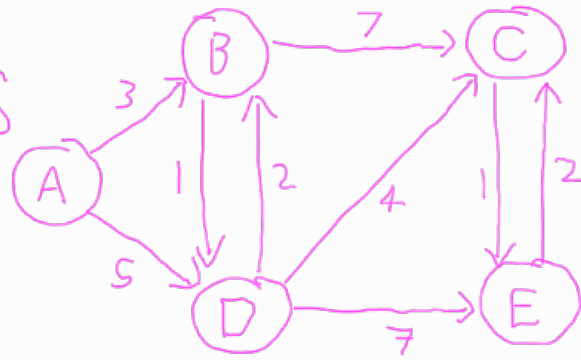
Dijkstra's Algorithm is a polynomial-time algorithm -  $O(V^2)$  - for finding the shortest path in a graph  $G=(V,E)$  from one vertex  $A \in V$  to every other vertex.

### Example

Let

$$\begin{pmatrix} 0 & 3 & 0 & 5 & 0 \\ 0 & 0 & 7 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 \\ 0 & 2 & 4 & 0 & 7 \\ 0 & 0 & 2 & 0 & 0 \end{pmatrix}$$

visualisation



be the adjacency matrix of a graph  $G$ .

Let  $A$  be the source vertex - first row + column.

Dijkstra's Algorithm outputs two arrays,  $d$  and  $\pi$ :

- $d$  contains the distance of the shortest path from the source to each vertex
- $\pi$  contains the predecessor node along this shortest path for each node.

We initialize these arrays by setting  $d(v) = \infty$  and  $\pi(v) = \text{Null}$  for every vertex, with the source having  $d(A) = 0$

|       | A    | B        | C        | D        | E        |
|-------|------|----------|----------|----------|----------|
| $d$   | 0    | $\infty$ | $\infty$ | $\infty$ | $\infty$ |
| $\pi$ | null | null     | null     | null     | null     |

Next, we initialize  $S$  to store the vertices that we have already found the shortest distance

Also initialize  $Q$  as a priority queue storing all other vertices, with the vertex  $v$  with smallest  $d(v)$  at the front.

$$S = \emptyset$$

$$Q = A, B, C, D, E$$

While  $Q$  is not empty, remove the vertex  $u$  s.t.  $d(u)$  is smallest and add it to  $S$ , and explore all edges leaving  $u$ , updating  $d$  and  $\pi$  for neighbouring vertices if the distance is shorter.

$$S = A, Q = B, C, D, E$$

$$\text{Edge } (A, B) = 3 \Rightarrow \text{Distance from } A \rightarrow B = 3$$

$3 < \infty$  so update  $\pi(B)$  and  $d(B)$

Edge  $(A,D) = 5 \Rightarrow$  Distance  $A \rightarrow D = 5$

$5 < \infty$  so update  $\pi(D)$  and  $d(D)$

|       | A    | B | C        | D | E        |
|-------|------|---|----------|---|----------|
| d     | 0    | 3 | $\infty$ | 5 | $\infty$ |
| $\pi$ | null | A | null     | A | null     |

When manually evaluating Dijkstra, we can ignore any edges going to any nodes in  $S$  - Since  $Q$  is a priority queue, any new path to a node that has already been explored must be longer.

|              |              |              |              |              |
|--------------|--------------|--------------|--------------|--------------|
| <del>0</del> | <del>3</del> | <del>0</del> | <del>5</del> | <del>0</del> |
| 0            | 0            | 7            | 1            | 0            |
| 0            | 0            | 0            | 0            | 1            |
| 0            | 2            | 4            | 0            | 7            |
| 0            | 0            | 2            | 0            | 0            |

B has the smallest  $d$ , so repeat this process with B next:

$S = A, B$        $Q = D, C, E$

$(B,D) = 1 \Rightarrow$  Distance  $A \rightarrow D = 1 + d(B) = 1 + 3 = 4$

$4 < 5$  so update  $\pi(D)$  and  $d(D)$

$(B,C) = 7 \Rightarrow$  Distance  $A \rightarrow C = 7 + 3 = 10$

$10 < \infty$  so update  $\pi(C)$  and  $d(C)$

|       | A    | B | C  | D | E        |
|-------|------|---|----|---|----------|
| d     | 0    | 3 | 10 | 4 | $\infty$ |
| $\pi$ | null | A | B  | B | null     |

|              |              |              |              |              |
|--------------|--------------|--------------|--------------|--------------|
| <del>0</del> | <del>3</del> | <del>0</del> | <del>5</del> | <del>0</del> |
| <del>0</del> | <del>0</del> | <del>7</del> | <del>1</del> | <del>0</del> |
| 0            | 0            | 0            | 0            | 1            |
| 0            | 2            | 4            | 0            | 7            |
| 0            | 0            | 2            | 0            | 0            |

Repeat with D:

$S = A, B, D$        $Q = C, E$

$(D,B) = 2 \Rightarrow A \rightarrow B = 4 + 2 = 6$

$6 > 3$  so don't update  $d$  and  $\pi$ .

Note: Since B is in  $S$  we know we have already found the shortest path to B.

$(D,C) = 4 \Rightarrow A \rightarrow C = 4 + 4 = 8$

$8 < 10$  so update  $d(C)$  and  $\pi(C)$

$(D,E) = 7 \Rightarrow A \rightarrow E = 4 + 7 = 11$

$11 < \infty$  so update  $d(E)$  and  $\pi(E)$

|       | A    | B | C | D | E  |
|-------|------|---|---|---|----|
| d     | 0    | 3 | 8 | 4 | 11 |
| $\pi$ | null | A | D | B | D  |

|              |              |              |              |              |
|--------------|--------------|--------------|--------------|--------------|
| <del>0</del> | <del>3</del> | <del>0</del> | <del>5</del> | <del>0</del> |
| <del>0</del> | <del>0</del> | <del>7</del> | <del>1</del> | <del>0</del> |
| 0            | 0            | 0            | 0            | 1            |
| <del>0</del> | <del>2</del> | <del>4</del> | <del>0</del> | <del>7</del> |
| 0            | 0            | 2            | 0            | 0            |

C:

$$S = A, B, D, C \quad Q = E$$

$$(C, E) = 1 \Rightarrow A \rightarrow E = 8 + 1 = 9$$

$9 < 11$  so update  $d(E)$  and  $\pi(E)$

|       | A    | B | C | D | E |
|-------|------|---|---|---|---|
| d     | 0    | 3 | 8 | 4 | 9 |
| $\pi$ | null | A | D | B | E |

|              |              |              |              |              |
|--------------|--------------|--------------|--------------|--------------|
| <del>0</del> | <del>3</del> | <del>0</del> | <del>5</del> | <del>0</del> |
| <del>0</del> | <del>0</del> | <del>7</del> | <del>0</del> |              |
| <del>0</del> | <del>0</del> | <del>0</del> | <del>0</del> | <del>1</del> |
| <del>0</del> | <del>2</del> | <del>4</del> | <del>0</del> | <del>7</del> |
| <del>0</del> | <del>0</del> | <del>2</del> | <del>0</del> | <del>0</del> |

In practice, when manually evaluating Dijkstra, we can stop here. But the algorithm does the last one anyway.

E:

$$S = A, B, D, C, E \quad Q = \emptyset$$

$$(E, C) = 2, \quad 9 + 2 = 11$$

$11 < d(C)$   $\Rightarrow$  no updates needed

|       | A    | B | C | D | E |
|-------|------|---|---|---|---|
| d     | 0    | 3 | 8 | 4 | 9 |
| $\pi$ | null | A | D | B | E |

|              |              |              |              |              |
|--------------|--------------|--------------|--------------|--------------|
| <del>0</del> | <del>3</del> | <del>0</del> | <del>5</del> | <del>0</del> |
| <del>0</del> | <del>0</del> | <del>7</del> | <del>0</del> |              |
| <del>0</del> | <del>0</del> | <del>0</del> | <del>0</del> | <del>1</del> |
| <del>0</del> | <del>2</del> | <del>4</del> | <del>0</del> | <del>7</del> |
| <del>0</del> | <del>0</del> | <del>2</del> | <del>0</del> | <del>0</del> |